

Physics-Based Learning and Control for Downwash in Multi-Quadrotor Systems

Anoop Kiran (ajkiran), Bahar Charyyeva (bcharyye), Maxwell Adorsoo (madorsoo), Zhao (Bill) Yan (zyan33)

CSCI1470 Deep Learning Final Project, Brown University

Contents

1	Introduction	2
2	Methodology	2
2.1	Quadrotor Dynamics	3
2.2	Downwash Effect	3
2.3	Proportional Integral Derivative (PID) Control	3
2.4	Model Architecture	4
2.5	Dataset	4
3	Results	5
3.1	PID Controller	5
3.2	MLP Prediction	5
4	Challenges	6
5	Reflections	7

1 Introduction

In this project, we implemented the framework proposed in the KNODE-MPC paper^[1], which aims to train two models: the KNODE model and the MPC controller. The KNODE model is designed to learn the residual dynamics between a first-principle-based physics model and the actual behavior of a drone during flight. This physics model predicts the drone’s vertical acceleration, as well as its pitch, roll, and yaw under various conditions. The MPC controller model, on the other hand, is responsible for computing the control inputs required for the drone to maintain a consistent flight path. Our implementation extends the original paper’s single-drone setting to a two-drone system. We also adapt the residual model to account for the *downwash effect*, the induced airflow generated below a quadrotor due to its rotating propellers, which affects neighboring vehicles when flying near its proximity.

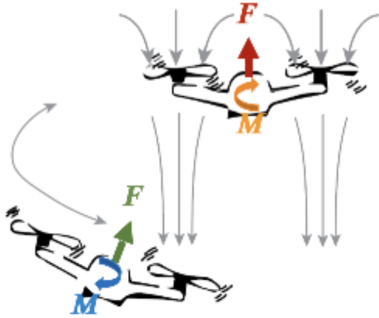


Figure 1: Schematic of Downwash Effect

The primary motivation for this extension

came from considering the broader real-world applications of quadrotors in deployment, as in Fig. 2. Since the paper already demonstrates its effectiveness on a single quadrotor, we were curious about how it will perform in multi-agent scenarios, such as drone swarms. Specifically, we wanted to explore how the air currents from an upper quadrotor would influence the dynamics and control of a lower one. This led us to model and analyze the interactions between two vertically aligned quadrotors. Moreover, our group implemented a simpler Proportional Integral Derivative (PID) controller and a physics-based Neural Net (NN) compared to the original paper’s KNODE-MPC framework. As this is a structured prediction problem - the model is attempting to predict the residual error from the first principles physics model - we predicted the force compensation by the lower quadrotor using PID and physics-based NN successfully, as discussed in Section 3.

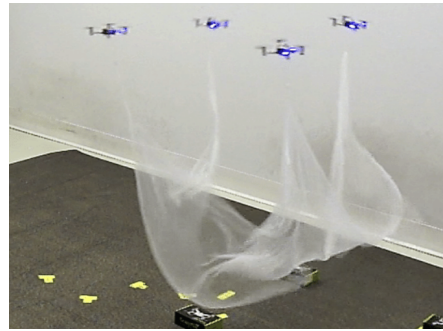


Figure 2: Real-World application of Multi-Quadrotor System^[2]

2 Methodology

2.1 Quadrotor Dynamics

Our implementation follows the quadrotor dynamics of the KNODE-MPC framework proposed in Chee et al.^[1]. If m and $J := \text{diag}(J_{xx}, J_{yy}, J_{zz})$ denote the mass and moment of inertias, respectively, then the equation of motion for the dynamics of the quadrotor can be stated as:

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} := \ddot{\mathbf{r}} = \frac{1}{m} \left(\begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + R_B^W \begin{bmatrix} 0 \\ 0 \\ u_1 \end{bmatrix} \right), \quad (1)$$

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} := \dot{\omega} = J^{-1} \left(\mathbf{u}_2 - \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times J \begin{bmatrix} p \\ q \\ r \end{bmatrix} \right), \quad (2)$$

where R_B^W is a transformation matrix between the quadrotor body frame and the world frame. u_1 is net motor forces, i.e., $u_1 = \sum_{i=1}^4 F_i$ and $\mathbf{u}_2$ is the moment (torque) vector, related to the motor forces by

$$\mathbf{u}_2 = \begin{bmatrix} 0 & L & 0 & -L \\ 0 & L & 0 & -L \\ \gamma & -\gamma & \gamma & -\gamma \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix} \quad (3)$$

where L is the arm length of the quadrotor and γ is the ratio between the moment and the trust coefficient of the motors. The orientation of the quadrotor in terms of Euler angles is related to the angular velocities by

$$\omega = \begin{bmatrix} \cos(\theta) & 0 & -\cos(\phi)\sin(\phi) \\ 0 & 1 & \sin(\phi) \\ \sin(\phi) & 0 & \cos(\phi)\cos(\theta) \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}. \quad (4)$$

Following the example of^[1], we define the state as $\mathbf{x} := [\mathbf{r} \ \dot{\mathbf{r}} \ \phi \ \theta \ \psi \ \omega] \in \mathbb{R}^n$, to have compact representation of the quadrotor system,

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \quad (5)$$

where $\mathbf{u} = [u_1, \mathbf{u}_2]^T \in \mathbb{R}^m$ and $f: \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ denotes a mapping given by 1, 2, and 4.

2.2 Downwash Effect

While KNODE framework^[1] learned the deviation between ground predictions and the theoretical prediction of (5), we introduced a second quadrotor to measure the downwash effect, the change in the direction of air deflected by the aerodynamic action of a rotor blade. Quadrotor lift follows Newton's third law of motion, where the force needed to deflect air downward is equal and opposite in direction to the lift force. The downwash effect becomes pronounced where two or more drones fly in proximity in a coordinated system. The downward deflection of the air of one drone could impact the trajectory of another drone.

We intend to leverage the KNODE model to predict and compensate for the aerodynamic disturbances in a two-quadrotor stacked flight configuration using a physics-based deep learning approach. Consider the scenario of two drones flying in parallel, separated by some height, with one drone above the other. The true system dynamics can be simulated using (5). However, due to the downwash effect of the upper drone, the trajectory of the lower drone will deviate from the true dynamics. This deviation can be modeled as

$$\epsilon = f_{dw}(\mathbf{x}, \mathbf{u}) - f_t(\mathbf{x}, \mathbf{u}), \quad (6)$$

where f_{dw} is the downwash disturbed system and the f_t is the true dynamics of the lower drone specified in (6). A deep neural network can then be used to minimize this positional deviation and compensate for the downwash effect.

2.3 Proportional Integral Derivative (PID) Control

The PID controller is a feedback-based closed-loop mechanism that compares a target value (setpoint, SP) with the desired values (process variable, PV), while automatically minimizing this error value^[4]. The error value is the difference between the setpoint and the pro-

cess variable, and the goal of the controller is to bring the process variable to the setpoint. Setting our downwash deviation as the error value of the controller, the true system dynamics as the setpoint, and downwash effect as the process variable, we rewrite (6) as

$$\epsilon = f_{dw}(\mathbf{x}, \mathbf{u}) - f_t(\mathbf{x}, \mathbf{u}) = SP - PV, \quad (7)$$

PID works to minimize the control variable using the input function

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad (8)$$

where $u(t)$ is the control variable, K_p , K_i , K_d are the proportional, integral, and derivative gains, respectively. Here $e(t)$ is the error value, t is the instantaneous time, and τ is the integration variable.

The proportional term has a large intermediate reaction to bring the process value to the setpoint, the integral term accumulates the improvements in the control variable, and the derivative term regulates overshoots and the rate of improvement in the control variable. Since the gains of the PID controller are multipliers, their optimal values can be learned using a deep neural network, which makes it a natural problem for deep learning.

2.4 Model Architecture

We started with the baseline model, which was a relatively shallow 2-layer MLP, as described in the paper. Our goal was to extend this to a deeper architecture to potentially improve prediction accuracy, especially in the more complex interaction scenario involving multiple quadrotors. So we expanded our current model to a 3-layer MLP with 64, 64, and 32 hidden layers, and the input being 12 units specified in Table 1

Inputs	Parameters
Rel. position (3)	$\Delta x, \Delta y, \Delta z$
Upper drone velocities (3)	$v_{x,u}, v_{y,u}, v_{z,u}$
Lower drone velocities (3)	$v_{x,l}, v_{y,l}, v_{z,l}$
Downwash parameters (3)	Strength (Γ), Width (w), Offset (δx)

Table 1: Neural network input parameters

ReLU and linear activation functions were used, along with the Adam optimizer. The model was trained to minimize the residual prediction error using Mean Squared Error (MSE) as the loss function. The three output parameters that the neural network produces would be the compensation force needed to counter downwash in the three axes about which we perform analyses (F_x, F_y, F_z).

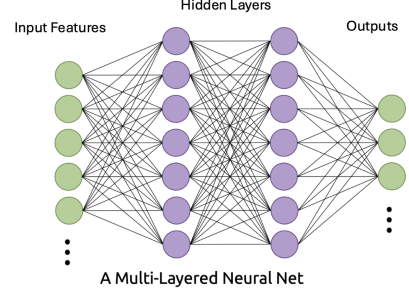


Figure 3: Simple Neural Net Diagram^[3]

2.5 Dataset

The data was obtained from simulations performed using quadrotor dynamics equations governed by Newton’s second law, outlined in Section 2.1. Given a certain set of states (e.g., position, velocity) and inputs (thrust of a quadrotor), we can predict its behavior (acceleration). Therefore, our data was generated based on these governing equations and reflects the system’s response under different conditions.

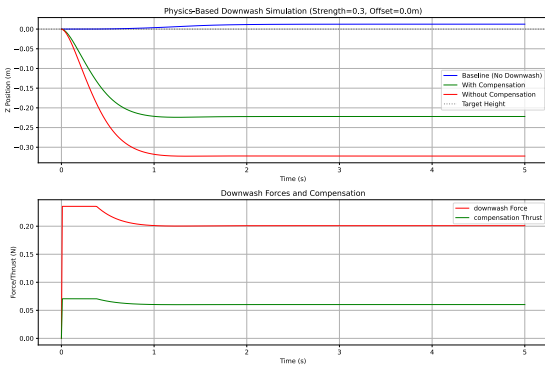
Data points were sampled at regular intervals from quadrotor flight trajectories. Each sample consists of a state-input vector (e.g., position, velocity, thrust), which was used to train the model. The residual model was then trained to capture disturbance effects, specifically the downwash caused by upper quadrotor influencing the lower drone.

3 Results

3.1 PID Controller

The hybrid approach, consisting of two components, the PID controller and the MLP residual prediction, was compared using two separate metrics. The first component of our project, the PID controller, was evaluated using comparisons of the downwash effect on drone flight with the controller and drone flight without the controller (baseline), as shown in Fig. 4. Trajectories with downwash showed a decrease in Z position of 0.2083 m and 0.3033 m, respectively. The effectiveness of our PID model was evaluated based on the % trajectory corrected, and the optimal hyperparameters occurred at $K_p = 20$, $K_i = 0.7$, $K_d = 10$, and Compensation = 10, with a 31.34% trajectory corrected value as pointed out in Table. 2, with the best percent improvement highlighted in yellow. Our selection of parameters with the initial guess already offered a relatively good estimate (30.9%), requiring us not to have to adapt much. Our parameter sweep led to the conclusion that increasing the K_i and decreasing the K_p lead to a slightly better improvement (31.34%). This is expected since decreasing the K_p helps attenuate the strict penalty on positional deviation, while smoothing out the final outcome through integration, with the slight increase in K_i .

Figure 4: Simple Neural Net Diagram



K_p	K_i	K_d	Comp.	Improv.
25	0.5	10	10	30.90%
20	0.7	10	10	31.34%
20	0.5	12	10	30.93%
20	0.5	10	12	30.95%
22	0.6	11	11	31.11%
18	0.4	10	9	30.79%

Table 2: PID parameter tuning

The animations that demonstrate how different strengths affect the motion can be accessed using this [link](#).

3.2 MLP Prediction

The second component of our model, the MLP, was evaluated using its ability to accurately predict forces in the x, y, and z directions of the drone. The optimal hyperparameter settings we used were 3 linear layers of 64, 64, and 32. The x and y axes had a noticeably low variance because no forces were being exerted on the drone in that plane, and there was some variance from the expected value on the z axis, which corroborates the force predictions shown in Fig. 3.2.

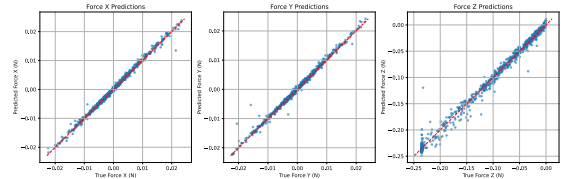


Figure 5: Physics-based Neural Net Force Predictions

The heatmaps in Fig. 3.2 (left) indicate the influence of downwash physical parameters strength (Γ) and horizontal offset (δx) on the overall observed effect of downwash. Compared to the baseline, which consists of a quadrotor experiencing downwash with no controller. It is noticeable that downwash

strength has a threefold influence on the effect felt by the lower quadrotor, comparing the strength magnitudes of 0.1 and 0.5. This is expected since a larger strength indicates a pronounced effect that is felt by the lower quadrotor compared to the baseline, which consists of no quadrotor.

With increasing lateral offset (δx) of the downwash jet relative to its location emanating from the quadrotor, we can see the lesser observed effect on the quadrotor. These percentages are in 2000% range as it shows the necessity of having a controller, which is the system that helps a drone maintain its altitude, without which it would barely hover in air under the influence of the downwash disturbance.

With lesser strength and increased lateral offset, the effect of downwash is felt much lesser, which makes it desirable for force prediction leading to improvements in force prediction as seen by Fig. 3.2 (right), proving that the effect of downwash has an inverse relationship with its impact on quadrotor performance.

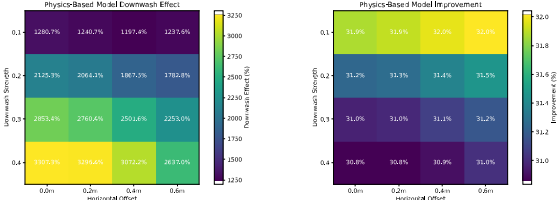


Figure 6: Heatmaps influenced by varying downwash parameters (inputs)

Bar charts in Fig. 3.2 compare the Root Mean Square (RMS)Error, a measure of model accuracy, across three scenarios: the baseline case without downwash effects (blue), which basi-

cally portrays the trajectory deviation from prescribed position of reference $0m$, the case with downwash effects but without compensation (red), and the case with both downwash effects and compensation (green). As predicted, there is a 31% improvement between the second (red) and third (green) charts, highlighting the effectiveness of the compensation mechanism.

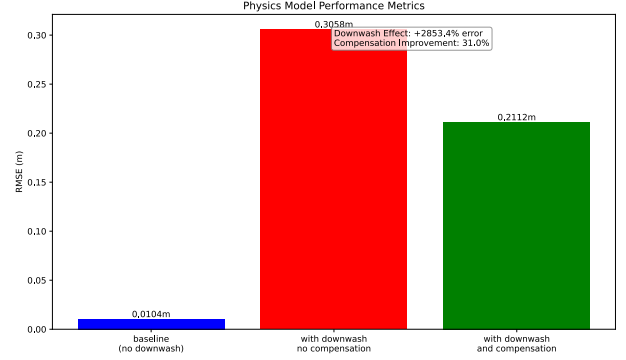


Figure 7: Physics Model Performance Metrics

Finally, the loss was calculated using Mean Squared Error (MSE) and is presented in Fig. 3.2 and it confirms that the training loss remains lower than the validation losses throughout the range of epochs tested:

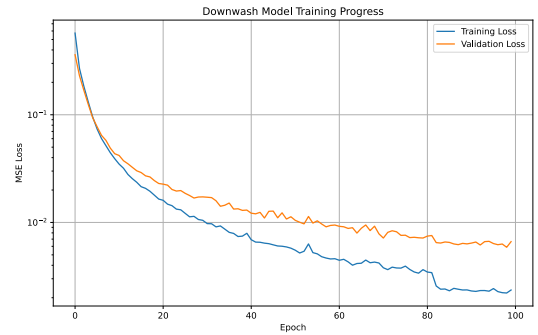


Figure 8: Training and validation losses

4 Challenges

We pivoted from our main goal of using KNODE-MPC because of model complexity and integration challenges. Given the timeline for the project, we preferred working with a drone controller that we were confident with and had sufficient documentation, which was PID. Although KNODE-MPC combines neural networks with first-principle models, forming a hybrid approach, it is inherently more complex, requiring more time and effort for debugging. Additionally, for the specific problem at hand, KNODE-MPC requires quality training data to learn the residual dynamics (like downwash effects). On the other hand, our physics-based approach using a feedforward neural network directly models these effects with analytical equations, avoiding the need for extensive data collection and training. To improve the predictions of the neural network, the MLP width and depth had to be increased. Decreasing test and training loss was correlated to increased network width and depth. Dropout allowed for less overfitting on

training data.

Additionally, thinking about our problem from an application perspective, we wanted to implement a model that would be generalizable to conditions far from the training data. Using the physics model ensures that we can capture the underlying principles of downwash, generalize better to new scenarios, and express adaptability, whereas KNODE-MPC might have struggled to perform well in conditions far from its training data.

Since the existing paper focuses on minimizing the residual of a single drone, we cannot directly compare our results to Chee et al., as there was no clear benchmark that defines success. But we were able to get a compensation improvement of around 31% using our physics-based approach, and the figures also clearly show a minimization in downwash effect with the trained model compared to the non-trained flight simulation, as some of the deviation caused by the additional drone is predicted by the feedforward neural network.

5 Reflections

Ultimately, our base, target, and stretch goals were different from what we had initially envisioned due to us pivoting from using the KNODE-MPC framework to the hybrid Physics NN + PID framework. Despite the changes, we were still able to reach our base and target goals by successfully running simulations for one, consequently for two quadrotors, and getting a high compensation improvement of 31.34%.

We pivoted from our main goal of using KNODE-MPC because of model complexity and integration challenges. Given the short amount of time for the project, we decided to use a more classic approach with a physics-based Neural Network for predictions and PID controller for compensation. Some benefits of our physics-based approach using a feedforward neural network included the fact that it

can directly model physics effects with analytical equations, instead of having two separate components like in KNODE. If we had more time to improve, we could have reached our stretch goal of obtaining the data from the physical experiments, or we would have found methods of fully integrating the KNODE-MPC framework and compared the performance differences one-to-one with the PID + physics-based NN approach.

The biggest takeaways from this project were that simpler and more general model frameworks, such as our physics-based neural network control models and PID, could sometimes outperform the more complex models like KNODE-MPC. This is essential because deployment on resource-constrained microcontrollers requires simpler models that are capable of performing optimization and gra-

dient calculations quickly onboard, which is especially the case for the Crazyflie quadrotors. Therefore, our goal was to limit computational complexity without compromising on the performance. Even though the PID controller was used due to the time constraint

of this project, the results showed that it was more effective. Lastly, collaboration and group skills, such as organizing meetings and making posters, were practiced throughout the group project.

References

- [1] Kong Yao Chee, Tom Z. Jiahao, and M. Ani Hsieh. KNODE-MPC: A Knowledge-Based Data-Driven Predictive Control Framework for Aerial Robots. *IEEE Robotics and Automation Letters*, 7(2):2819–2826, 2022.
- [2] Ryan Cotsakis, David St-Onge, and Giovanni Beltrame. Decentralized collaborative transport of fabrics using micro-uavs. In *International Conference on Robotics and Automation (ICRA)*, pages 7734–7740, 2019.
- [3] Eric Ewing. CSCI1470: Deep Learning, 2025. Last accessed 2 May 2025.
- [4] Scott Zhuge. PID “Proportional, Integral, and Derivative” Control Theory, Aug 2020.